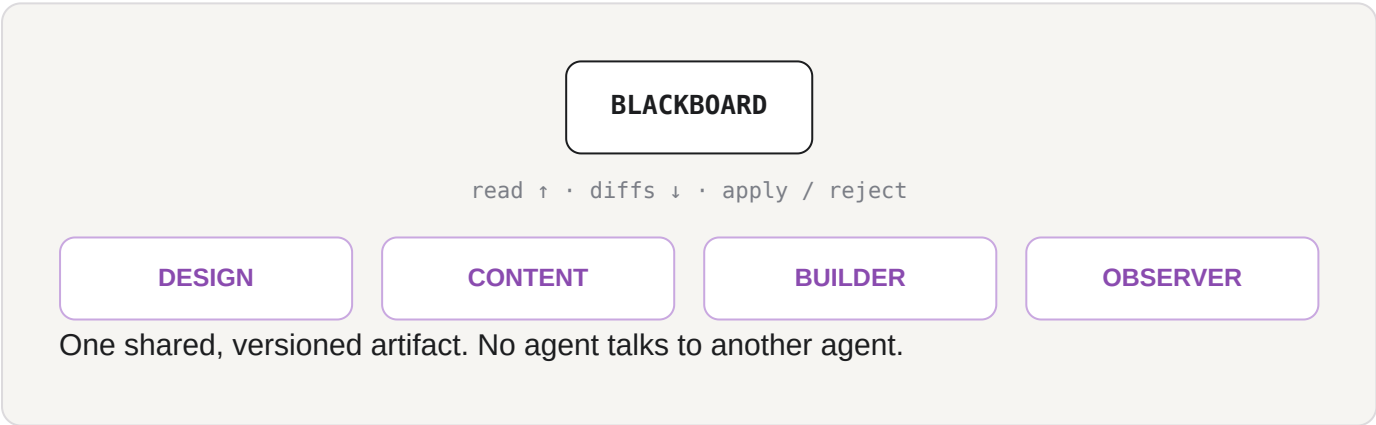


Website Builder Harness

An agent harness that builds business websites end to end from a prompt — by refusing to guess at the two things that actually determine whether the output is usable: what the site is for, and what real material it has to work with.



DESIGN PHASE — NO CODE WRITTEN

Scoped to B2B SaaS landing pages for v1

Last updated 2026-08-21
Report generated 2026-08-25

CONTENTS

Table of Contents

01	What This Is	The product, in one paragraph
----	--------------	-------------------------------

02	Product Thesis	Goal elicitation & real content
03	How a User Experiences It	Three gates, walked end to end
04	System Architecture	The blackboard, not an agent mesh
05	The Agents	Design, content, builder, observer
06	Goal Memory	A document, not an index
07	Sources & References	Primary reference vs. checklist
08	Images	Regeneration, treatment, PII scrub
09	Technology Stack	Fixed, deliberately
10	V1 Vertical Slice	What actually gets built first
11	Open Questions	What is still undecided

01 – OVERVIEW

What This Is

A system, not a model: an agent harness that takes a business owner's prompt and produces a real, launch-ready website — not a demo.

The harness is scoped deliberately narrow: business websites. B2B SaaS landing pages, portfolios, local-service sites, small commerce. It does not try to be a general website generator, and that narrowness is what lets it make stronger assumptions everywhere else in the pipeline — one fixed tech stack, one blackboard shape, one review loop.

Comparable products — Lovable, Emergent — take a one-line prompt and guess at everything else: what the business does, who it's for, what it looks like, what content goes on the page. The output is fast and looks plausible in a demo, and is unusable the moment a real business tries to ship it, because it's built on invented facts.

This harness refuses to guess at the two things that actually determine whether a generated site is usable: **what the site is for**, and **what real material it has to work with**. Everything else in the design — the blackboard, the agent split, the stack — is infrastructure built in service of protecting those two things from drifting or fading out over a long generation run.

Where this document sits

The project is currently in the **design phase** — no code has been written yet. This report exists to give a complete, standalone picture of the architecture and the decisions behind it: what's fixed, what's deliberately deferred, and what's still an open question.

02 — PRODUCT THESIS

Two Differentiators

Everything else in this system is infrastructure in service of these two ideas.

Goal Elicitation Before Generation

The run starts by pinning down intent before anything is generated. Open-ended input like "I want a website" gets autocomplete chips — B2B SaaS platform, portfolio, local service — that expand into a structured brief. A user who arrives with a detailed goal already written skips straight ahead. Nothing downstream runs until the brief exists.

Real Content, Not Filler

Every AI-built business site dies on lorem ipsum and stock photos. This system asks the user for their actual material — copy, and especially product or dashboard imagery — and shows source snippets as reference, so the user understands what's being asked for and why. This is the single most valuable idea in the design, and the least solved anywhere else in this category.

Why not just guess, then let the user edit?

Because editing a wrong foundation costs more than answering three questions up front. A site generated on an unstated audience or a fabricated product shot doesn't fail loudly — it fails quietly, in tone and content, in a way a business owner often can't articulate but can always feel. Front-loading the brief and the real material converts an open-ended editing problem into a bounded generation problem.

03 – USER EXPERIENCE

How a User Experiences It

Three human gates, not fifteen. Everything between them runs autonomously.

An earlier version of this design had fifteen-plus approval points — skeleton, design, stack, content, every element, final. Nobody finishes a flow like that. The current design collapses review down to the three moments where a business owner's judgment is actually load-bearing, and lets the agents run unattended everywhere else.

1. The Brief

HUMAN GATE

The user describes what they need. Vague input ("I want a website") is met with structured chips that expand into a full brief; a detailed goal typed up front is parsed directly. A small "librarian" step keeps refining this brief as the user says more — adding, revising, or superseding earlier statements — for the rest of the run.

Source gathering

AUTONOMOUS

The user supplies one or more reference sites. Playwright extracts DOM/accessibility structure and full-page screenshots. Sources are matched against the brief before anything downstream uses them.

2. Design Direction

HUMAN GATE

The design agent proposes which reference site's skeleton best fits the brief, shown with a snippet of that source, and its intended palette and structure. The user approves or redirects before any building starts.

Content interview & build

AUTONOMOUS

The content agent asks for the user's real copy and product imagery, showing source snippets as reference for what's being requested, and drafts content the user can react to rather than a blank prompt. The builder then constructs the site section by section, screenshotting and fixing its own output as it goes.

3. Full-Page Preview

HUMAN GATE

The observer has already reviewed the complete site against the brief, the approved design decisions, and the sources. The user sees a finished preview URL, not a checklist of pass/fail criteria.

ESCALATION, WHEN SOMETHING NEEDS A DECISION

Any mid-run ambiguity is surfaced as a concrete choice, not a dump of failed criteria. *"This section can go with a wider hero or a tighter two-column layout — which do you want?"* is answerable by a business owner. `contrast_ratio_failed: 3.9` is not.

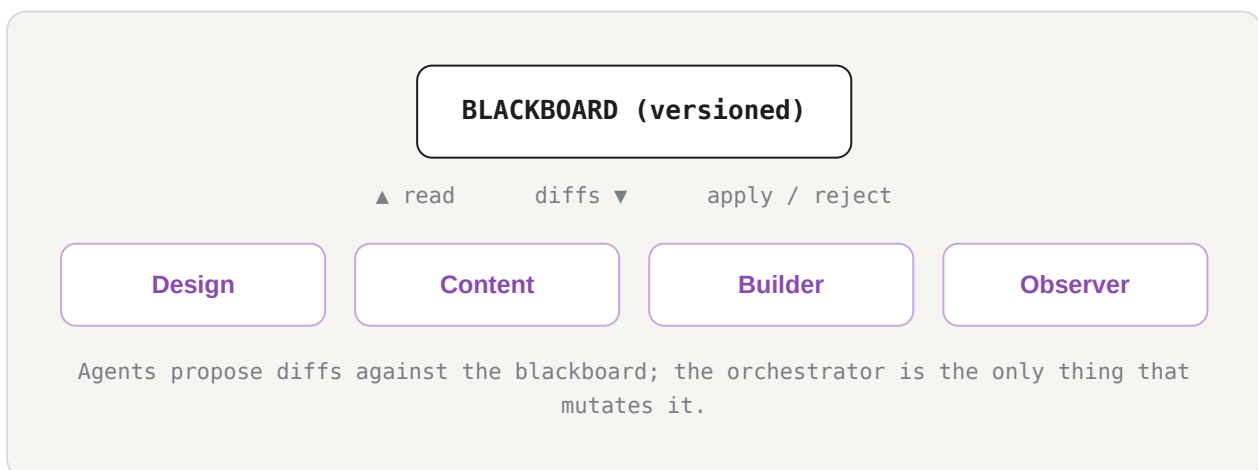
Internally, iterative loops (build → look → fix, critique → refine) are capped at **three attempts**. Past that, the pattern plateaus and a judge model starts inventing objections to justify another pass — so the system stops and either accepts the result or escalates to the user.

04 — ARCHITECTURE

System Architecture: Blackboard, Not Agent Mesh

No agent talks to another agent. Every state change goes through one shared, versioned artifact.

Free-form agent-to-agent communication produces unbounded loops, context explosion, and — critically for a system a business is trusting with its website — no way to debug why an output changed. The alternative used here: a single shared, versioned artifact called the **blackboard**. Agents read it and propose **diffs**. An orchestrator applies or rejects each diff. Every transition is recorded, so any run is replayable and any regression traces back to the exact diff that caused it.



Blackboard shape

Stored as Postgres JSONB. Backend is FastAPI + Postgres + Docker with a job queue, Redis for queue and state.

FIELD	HOLDS
brief	Structured goal — audience, category, tone, offering, hard requirements
constraints	Short verbatim strings, injected into every agent prompt unchanged
sources	Primary reference + secondary checklist, with extraction artifacts
design_system	The design agent's recorded decisions — palette, type scale, spacing, radii, shadows, imagery treatment
sitemap	Pages and section order
sections[]	Per-section state: blueprint, content, assets, build status
assets	Uploaded originals and derived variants
content	Copy per section, with provenance — user-supplied vs. drafted
decisions	Append-only log with supersede traces

05 – THE AGENTS

Four Agents, One Blackboard

Each reads the blackboard, proposes diffs, and does one job.

Design Agent

Looks at sources and the brief and proposes a direction: which source's skeleton best aligns with the goal, shown with a snippet of that source, for user approval. Then owns palette, page count, skeleton, architecture, and imagery treatment — the calls a real designer makes.

Content Agent

Studies what content the sources use, checks it against the brief, and asks the user for their real material — showing source snippets as reference for what's being requested. Generates drafts to react to rather than blank prompts to fill, and keeps provenance on every piece of copy: user-supplied vs. drafted.

Builder Agent

Builds section by section. After building, it visits its own output through the browser — screenshots, accessibility tree, clicking through — and fixes what's broken or off-spec before moving to the next section. This build → look → fix loop is what pure code-gen misses.

Observer

Reviews the full site against the brief, the approved decisions, and the sources. Approves or rejects with reasons — the last check before the user sees the preview.

Design authority

The design agent's output is the source of truth. There is no external rulebook constraining what it may propose — a rulebook would cap the output at the rulebook's own taste, and the model is better than that.

The distinction that matters: rules **upstream** of the design agent (a rulebook defining "good design") are rejected outright. The design agent's own recorded decisions, **downstream**, are not a constraint system — they're a transcript. The palette and spacing it chose get written down so the *builder* is held to them: not "you must use an 8-point scale," but "you chose these nine colors; you don't get a tenth at section six."

WHY THIS MATTERS

Without a recorded transcript, "the design agent is the source of truth" has no enforcement mechanism — the builder is a separate model call that drifts quietly, and the drift shows up as the page falling apart in its lower third. Model capability decides *what*; the recorded decision enforces *consistency*.

Judging follows the same logic. The observer's useful job is catching what the design agent didn't anticipate — a brand color that kills contrast, content longer than the layout assumed, a mobile break — not scoring taste. A judge prompted with "is this good design?" approves anything on the first pass and nitpicks randomly on the third.

A Document, Not an Index

The goal must not fade out of context over a long run — but vector embeddings are the wrong tool for this specific problem.

RETRIEVAL FAILS BY OMISSION

A brief is a few thousand tokens — small enough to inject in full, always, into every agent. Retrieval adds nothing and introduces a silent failure mode: the design agent queries "color direction," and the line where the user said *"don't use blue, our competitor is blue"* doesn't come back, because it embeds as competitor-talk, not color-talk. The constraint doesn't error — it's just absent.

APPEND-ONLY STORES CAN'T HANDLE CONTRADICTION

The user says "friendly and approachable" in the brief, then forty minutes later, looking at a draft: "actually more serious, we sell to banks." That's a *replacement*, not an addition. A vector store holds both; both retrieve; both look equally relevant; the agent averages them into mush.

The model used instead

- **The brief is a structured document, injected whole.** A small **librarian** step runs whenever the user says something new: it reads the statement plus the current brief and emits a diff — add, revise, or **supersede**. Superseded items keep a one-line trace of what they were and when they changed, which is what makes tone drift debuggable and the run replayable.
- **Hard constraints are a separate verbatim list.** Short strings — "no blue," "must mention SOC 2," "no stock photos of people." Appended in full to every agent prompt. Never summarized, never compressed, never retrieved-maybe. This is the actual anti-fade mechanism, and it stays small by design.
- **Embeddings earn their place on the sources, not the goal.** Full DOM plus screenshots of several sites genuinely is too big to inject, so that's what gets retrieved against. Later, the same applies to a blueprint library and past builds across projects.

THE SPLIT, IN ONE LINE

The goal is small and must be complete; the sources are big and can be sampled.

A memory graph is more defensible than embeddings if relations need modeling — *this section exists because of that decision* — but for a single site the relation set fits in named JSON fields. That's revisited only once a brief change must propagate across a twelve-page site, which is out of scope for v1.

07 — SOURCES

Sources & References

Reference sites are used in two different roles, not blended into one pile.

ONE PRIMARY REFERENCE

Skeleton, rhythm, section order, pacing.
Blending six sites yields the mean of six sites, which is generic by definition — design is not additive.

THE REST AS A CHECKLIST

What sections exist in this category at all:
pricing table, logo wall, integration grid,
comparison table, FAQ.

Extraction runs through **Playwright**, not Selenium — faster, a better screenshot API, and an accessibility tree that's far more useful to a judge model than raw DOM. Both the DOM/a11y structure and full-page screenshots are captured, so the system sees a site the way a user does, not only the way markup describes it.

Sources are matched against the goal — do these actually serve this brief? — before anything downstream uses them.

LEGAL BOUNDARY — ENCODED IN SYSTEM PROMPTS, NOT A POLICY DOC

Extracting structural and layout patterns is fine. Closely reproducing a specific company's distinctive look for their direct competitor is not. This distinction lives in the prompt, where it's actually enforced — not in a policy document nobody reads.

08 — IMAGES

Images

The user's real imagery — product and dashboard screenshots — is treated as primary material, not decoration.

Regeneration

In scope. Frontier image models now handle precise UI regeneration well enough to restyle a raw screenshot to match the winning source's presentation. The practical safeguard built alongside it: a **vision diff** between the original and the regenerated version, so altered numbers, garbled labels, or invented UI surface before the user sees them — not after.

Presentation treatment

This carries most of the perceived visual quality, independent of regeneration. What makes a dashboard shot look designed is the treatment around it: browser chrome or device frame, perspective tilt, soft shadow, gradient backdrop, bleeding off the right edge, a masked fade, a zoomed detail crop with a callout. A vision pass characterizes how the primary source presents its own product imagery, and the builder reproduces that treatment — this is CSS/SVG applied over the asset, not a new image.

Image generation also covers the layers around the content: backdrops, gradients, texture, spot illustration, empty-state art, icons — plus straightforward edits like background removal, upscaling, recolor of flat illustrations, and dark-mode variants.

Two things cheap now, painful to retrofit

PII Scrub

Dashboard screenshots are full of real customer names, emails, revenue figures. A vision pass flags and blurs or substitutes them — this fits the real-content ethos exactly, and nobody else in this space has it.

Asset Variants as First-Class

One upload becomes hero-wide, card-square, mobile-crop, and dark variant. Decided early, or the builder starts stretching things to fit slots it was never given the right shape for.

GAP CASE

A pre-launch user has no product to screenshot. This needs a standing policy — abstract or illustrative hero rather than a fabricated product shot — decided once, up front, not improvised per run.

Stack: Fixed, Deliberately

Quality in this category comes precisely from the stack being fixed — the model never spends reasoning budget on architecture, and every pattern it generates is one it has seen ten thousand times.

LAYER	CHOICE
Site output	Astro or Next, Tailwind, shadcn/ui, one CMS, one form/email provider
Harness	FastAPI, Postgres (JSONB blackboard), Redis, Docker, job queue
Browser	Playwright

DELIBERATELY EXCLUDED: AN EVOLVING TECH-STACK AGENT

An agent hunting for the newest framework picks something past the model's knowledge cutoff, generates confident wrong code from a docs page it half-remembers, and fails in ways nothing downstream can diagnose. The modern edge lives in curated, versioned **section blueprints** — owned, tested, swappable — not in framework selection at runtime.

10 – SCOPE

V1 Vertical Slice

One category, end to end: B2B SaaS landing pages.

-
1. Goal elicitation → structured brief JSON

2. One reference URL, manually pasted → Playwright screenshot + section-structure extraction

3. Design agent → design decisions + sitemap → one human approval

4. Builder → Astro + Tailwind, section by section from the blueprint library



5. Build → screenshot → review → fix loop

6. Preview URL

DEFERRED TO V2 – AND ONLY ONCE V1 HAS VISIBLY FAILED SOMEWHERE

Multi-agent coordination beyond the four core agents, multi-source synthesis, a memory graph, multi-page brief propagation, and blueprint retrieval across past builds.

11 – STATUS

Open Questions

What's still undecided, as of this report.

- **The content interview's actual interface.** "Ask the user for real content" is a UI problem, not an agent problem — a business owner who won't write an About page also won't fill in twelve prompts. Voice input? A three-question ladder per section with a generated draft as the starting point to edit? This is the differentiator described in §2, and currently the least designed part of the system.
- **Where the primary-reference choice sits.** Agent-proposed with a snippet for user approval (the current plan) vs. letting the user pick the primary reference outright.
- **Policy for the pre-launch / no-assets user.** Referenced in §8 as a gap case; needs a standing default, not an improvised per-run choice.
- **Where the harness repository lives.** The project-context file this report is built from currently sits in a scratch directory; it belongs at the root of the real project.

Current status

Design phase. No code has been written. The architecture above — the blackboard shape, the four-agent split, the fixed stack, the three-gate interaction model — represents decisions made before implementation begins, so that early build work has a fixed target rather than one that shifts under it.

